Erreurs et mini-projet

Bloc 3 : comprendre les exceptions, les attraper avec try et except, gérer les fichiers manquants et les données invalides. Bloc 4 : construire un analyseur de données complet, qui assemble tout ce qui a été vu depuis le Day 1.

1. Qu'est-ce qu'une exception ?
2. Les trois erreurs du jour
3. try et except
4. else, finally et as
5. Gérer un fichier manquant
6. Gérer des données invalides
7. Lever une erreur : raise
8. Exercices du bloc 3
9. Mini-projet : l'analyseur de notes
10. Grille de validation
11. À savoir par cœur

---

1. Qu'est-ce qu'une exception ?

Définition 1.1 *(exception)*

Une **exception** est un signal d'erreur émis par Python **pendant l'exécution**, quand une instruction ne peut pas être réalisée. Chaque exception a un **type** (`ValueError` ...) et un **message**.

Proposition 1.2 *(exception non attrapée)*

Si une exception n'est pas attrapée, le programme **s'arrête** et affiche un **traceback** : la trace des lignes qui ont mené à l'erreur. Il se lit de bas en haut : type, message, puis ligne.

Remarque 1.3 *(erreur de syntaxe ou exception)*

Une `SyntaxError` ou une `IndentationError` survient **avant** l'exécution : le code est mal écrit et ne peut pas démarrer. Aucun `try` ne peut l'attraper, il faut corriger le code. Les exceptions de ce bloc surviennent **pendant** l'exécution, sur un code bien écrit, à cause des **données**.

2. Les trois erreurs du jour

EXCEPTION	SIGNIFIE	EXEMPLE QUI LA PROVOQUE	IMAGE
`ValueError`	bon type, mais valeur impossible	`float("abc")`	un papier où l'on a écrit « abc » à la place d'une note
`TypeError`	opération impossible entre ces types	`"18" + 2`	additionner un papier et des bonbons
`FileNotFoundError`	le fichier n'existe pas à ce chemin	`open("absent.txt")`	chercher un cahier dans un tiroir vide

Il en existe d'autres que tu as déjà croisées : `NameError` (nom inconnu), `IndexError` (case inexistante), `KeyError` (clé absente d'un dictionnaire), `ZeroDivisionError` (division par zéro).

3. `try` et `except`

Définition 3.1 *(try / except)*

```
try:
    instructions_risquees
except TypeDErreur:
    instructions_de_secours
```

En clair : `try` = « essaie ». `except` = « sauf si ça rate de cette façon-là, alors fais plutôt ça ». C'est le filet sous le trapéziste.

Proposition 3.2 *(déroulement)*

Python exécute le bloc `try` ligne par ligne. Si aucune erreur ne survient, le bloc `except` est ignoré. Si une erreur survient, **le reste du bloc try est abandonné** et Python saute dans le premier `except` dont le type correspond. Ensuite, le programme continue normalement après le bloc.

```
try:
    note = float("abc")      # ValueError ici...
    print("jamais affiché")  # ...donc cette ligne est sautée (Proposition 3.2)
except ValueError:
    note = 0
print(note)                  # 0 : le programme a continué
```

Corollaire 3.3 *(type non prévu)*

Si l'erreur n'est pas du type indiqué dans `except`, elle n'est pas attrapée : le programme s'arrête comme d'habitude (Proposition 1.2).

Méthode 3.4 *(plusieurs except)*

```
try:
    resultat = 10 / int(texte)
except ValueError:
    print("Ce n'est pas un nombre")
except ZeroDivisionError:
    print("Division par zéro")
```

Piège 3.5 *(le filet trop grand)*

`except:` tout seul, ou `except Exception:` sans rien afficher, attrape **toutes** les erreurs, y compris tes propres fautes de frappe. Le bug devient invisible. On attrape uniquement l'erreur **attendue**, et on met le moins de lignes possible dans le `try`.

4. `else`, `finally` et `as`

Définition 4.1 *(forme complète)*

```
try:
    ...                          # le code risqué
except ValueError as erreur:     # « as erreur » : on garde l'erreur pour li
    print(f"Problème : {erreur}")
else:
    ...                          # seulement si AUCUNE erreur n'est survenue
finally:
    ...                          # TOUJOURS exécuté, erreur ou pas
```

5. Gérer un fichier manquant

Méthode 5.1

```
def lire_texte(chemin):
    try:
        with open(chemin, encoding="utf-8") as f:
            return f.read()
    except FileNotFoundError:
        print(f"Fichier introuvable : {chemin}")
        return ""                    # une valeur neutre, et le programme con
```

Dans ExamPredict

Un lycéen dépose 20 PDF, dont un corrompu. Sans filet, tout s'arrête et les 19 autres ne sont jamais analysés. Avec `try` / `except`, on affiche « fichier ignoré » et on continue.

6. Gérer des données invalides

Méthode 6.1 *(valider ligne par ligne)*

On traite chaque ligne **séparément**. Une ligne invalide est mise de côté, avec sa raison, au lieu de faire planter tout le fichier.

```
valides, rejets = [], []
for brut in ["12", "15.5", "abc", "", "18"]:
    try:
        valides.append(float(brut))
    except ValueError:
        rejets.append(brut)
# valides = [12.0, 15.5, 18.0]     rejets = ['abc', '']
```

Dans le métier

Les vraies données sont toujours sales. Un ML Engineer ne supprime pas les rejets en silence : il les **compte** et les **rapporte** (« 312 lignes rejetées sur 10 000 »). Si 40 % des lignes sont rejetées, c'est le signe d'un vrai problème dans les données.

7. Lever une erreur : `raise`

Définition 7.1

`raise ValueError("message")` déclenche volontairement une exception. On l'utilise quand une donnée est techniquement lisible, mais impossible selon les règles du problème.

```
def verifier_note(note):
    if note < 0 or note > 20:
        raise ValueError(f"note hors limites : {note}")
    return note

try:
    verifier_note(22)
except ValueError as e:
    print(e)                    # note hors limites : 22
```

> **À dire**
>
> « try, c'est le trapéziste. except, c'est le filet. Et raise, c'est l'arbitre qui siffle quand une donnée est impossible. Une IA reçoit des données sales tous les jours : sans filet, elle tombe à la première erreur. »

---

8. Exercices du bloc 3

Fichier : `WEEK1/DAY3/exercices/b3_erreurs.py`.

Exercice 3.A · Nommer l'erreur ★

Quelle exception provoquent `int("douze")`, `"a" * "b"`, `open("x.txt")` (fichier absent) et `[1, 2][5]` ?

Corrigé. `ValueError`, `TypeError`, `FileNotFoundError`, `IndexError`.

Exercice 3.B · Prédire ★★

```
try:
    x = int("7")
    y = int("sept")
    print("A")
except ValueError:
    print("B")
print(x)
```

Corrigé. `B` puis `7`. x vaut 7, puis `int("sept")` lève une ValueError : « A » est sauté, et Python passe à l'except (Proposition 3.2). x a gardé sa valeur.

Exercice 3.C · Conversion sûre ★★

Écrire `en_nombre(texte)` qui renvoie le nombre, ou `None` si la conversion échoue.

```
def en_nombre(texte):
    try:
        return float(texte)
    except ValueError:
        return None

assert en_nombre("12.5") == 12.5
assert en_nombre("abc") is None
```

Exercice 3.D · Moyenne protégée ★★★

Écrire `moyenne_sure(notes)` qui renvoie la moyenne, ou 0 si la liste est vide, sans `if`.

```
def moyenne_sure(notes):
    try:
        return sum(notes) / len(notes)
    except ZeroDivisionError:      # len([]) vaut 0
        return 0
```

---

9. Mini-projet : l'analyseur de notes

Cahier des charges

Le fichier `projet/notes_classe.txt` contient une ligne par élève, au format `nom;note`. Certaines lignes sont invalides. Le programme `projet/analyseur.py` doit :

1. lire le fichier, et afficher un message clair s'il n'existe pas ;
2. valider chaque ligne : deux morceaux séparés par `;`, une note numérique, comprise entre 0 et 20 ;
3. calculer : effectif valide, moyenne, note max (avec le nom), note min (avec le nom), nombre d'admis (≥ 10) ;
4. afficher un rapport lisible, avec la liste des lignes rejetées et leur raison ;
5. sauvegarder le rapport dans `projet/rapport.json` ;
6. passer les tests `assert` fournis en bas du fichier.

L'architecture imposée : une fonction par tâche (Méthode 4.1 de la F1)

FONCTION	REÇOIT	RENVOIE	NOTIONS
`lire_lignes(chemin)`	un chemin	la liste des lignes non vides, nettoyées	`with open`, `strip`, `FileNotFoundError`
`analyser_ligne(ligne)`	`"Lina;14"`	`("Lina", 14.0)`, ou `raise ValueError`	`split`, `float`, `raise`
`charger_notes(chemin)`	un chemin	`(valides, rejets)`	`try` / `except ValueError as e`
`statistiques(valides)`	une liste de `(nom, note)`	un dictionnaire de résultats	`return`, la tirelire, `max` / `min` avec `key`
`sauver_json(donnees, chemin)`	un dict et un chemin	rien (écrit le fichier)	`json.dump`
`afficher_rapport(stats, rejets)`	les résultats	rien (affiche)	le **seul** endroit avec des `print`

Le résultat attendu

```
=== RAPPORT DE LA CLASSE ===
Élèves valides : 6
Moyenne        : 12.67
Meilleure note : Chloé (17.0)
Plus basse     : Jules (8.0)
Admis          : 4 / 6
Lignes rejetées : 4
  - Yanis;abc → note non numérique
  - Sarah; → note non numérique
  - Tom;22 → note hors limites : 22.0
  - mauvaise ligne sans point-virgule → format attendu nom;note
✅ Mini-projet validé
```

Méthode 9.1 *(l'ordre de construction)*

On construit et on teste **une fonction à la fois**, dans l'ordre du tableau. Après chaque fonction, on lance le fichier : les tests de cette fonction doivent passer avant d'écrire la suivante. C'est ainsi que travaillent les ingénieurs : de petits pas vérifiés.

Indice pour max et min avec le nom

`max(valides, key=lambda p: p[1])` renvoie le couple `(nom, note)` qui a la plus grande note. `p[1]` désigne la note dans chaque couple (fiche F2.DAY1, section 3).

10. Grille de validation du mini-projet (sur 20)

CRITÈRE	POINTS
Le fichier est lu avec `with open`, et un fichier manquant donne un message clair	3
`analyser_ligne` refuse les 3 cas invalides avec `raise ValueError`	4
Les rejets sont gardés avec leur raison, sans planter	3
Statistiques justes (moyenne, max, min, admis)	4
`rapport.json` est créé et relisible	2
Une fonction = une tâche, `print` seulement dans `afficher_rapport`	2
Tous les tests `assert` passent	2

11. À savoir par cœur

1. Une exception est une erreur pendant l'exécution. Non attrapée, elle arrête le programme.
2. `ValueError` = valeur impossible. `TypeError` = types incompatibles. `FileNotFoundError` = fichier absent.
3. Une erreur dans `try` fait sauter la suite du `try` et mène au `except` correspondant.

4	On attrape l'erreur précise attendue, avec le moins de lignes possible dans le `try`.
5	`except X as e` garde le message. `else` : s'il n'y a pas eu d'erreur. `finally` : toujours.
6	`SyntaxError` et `IndentationError` ne s'attrapent pas : on corrige le code.
7	Des données sales se traitent ligne par ligne : les valides d'un côté, les rejets avec leur raison de l'autre.
8	`raise ValueError("…")` refuse une donnée impossible selon les règles.
9	On construit un projet une fonction à la fois, testée avant de passer à la suivante.
